Testing (2)

CSE 362
Software Development and Professional Practice
Winter 2023
© Royal Military College of Canada

The F.I.R.S.T Rules

- **F**ast tests are more likely to be run as often as they should be.
- **I**ndependent tests will not depend on each other. You should be able to run tests in any order.
- **R**epeatable tests can be run in any hardware/OS environment.
- **S**elf-validating tests either pass or fail.
- **T**imely tests follow the three laws of TDD.

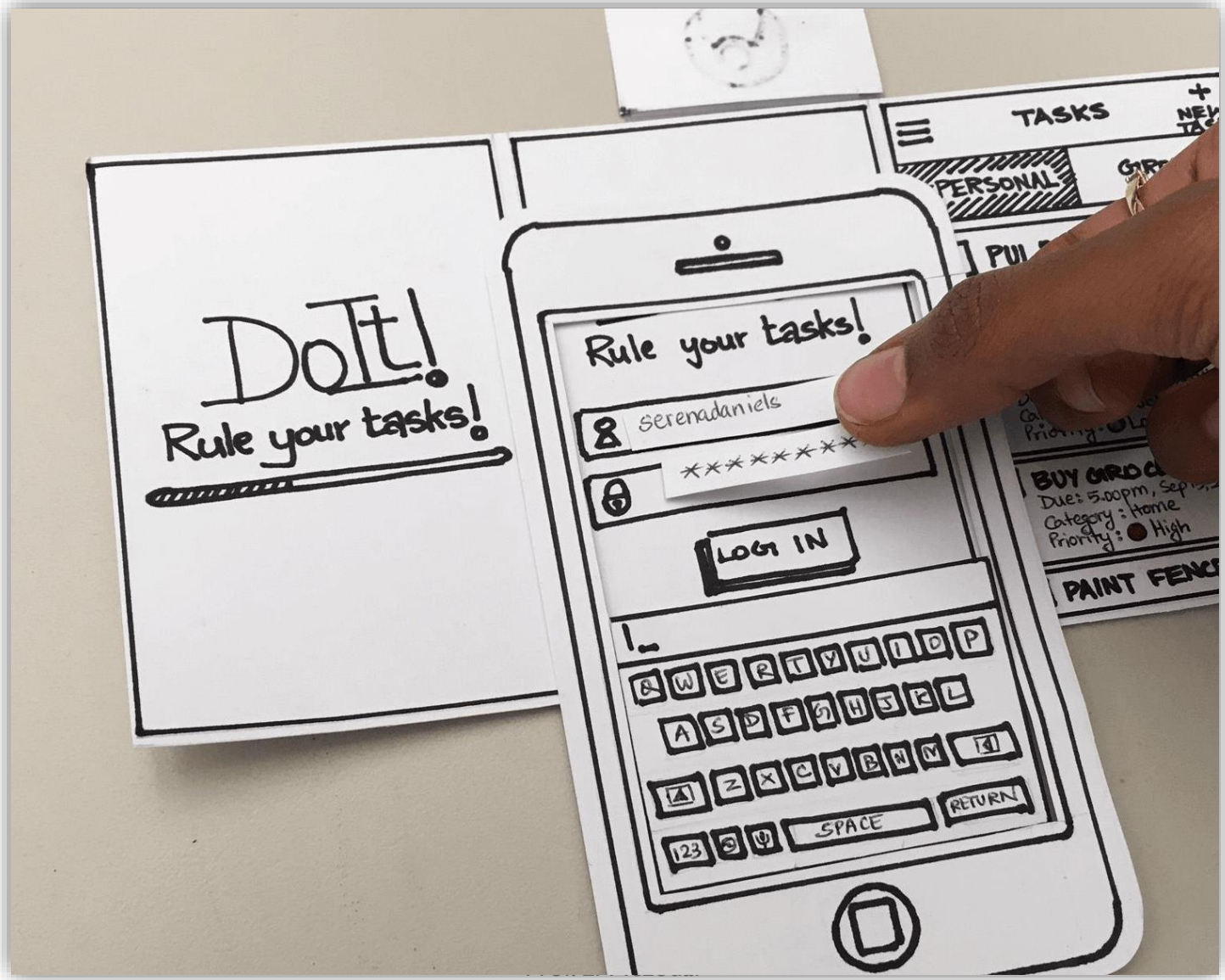
Overview of the Testing Process

Process	**Testing**
Coding	Unit Testing
Design	Integration Testing
Requirements Analysis	Validation Testing
System Engineering	System Testing

Overview, Cont.

- Test Planning – Allocate and schedule resources. Do this early on – as soon as models stabilize.
- Usability Testing – Test user interface. (See "Paper Prototyping", for example.)
- Unit Testing – Test individual methods and classes.
- Integration Testing –
  - First separate classes by threads.
  - Start by identifying independent classes and test them first, then test dependent classes, layer by layer.
  - Cluster testing – test a bunch of collaborating classes together.
  - Use regression testing, stub and driver code as needed (more about these later!).

Aside – Paper Prototyping



Overview, Cont.

- System Testing – Test all the classes together.
  - –Validation Testing tests the requirements listed in the RAD and user manual.
  - – Performance Testing checks non-functional requirements and additional design goals from the SDD. (Done by developers).
  - – Acceptance and Installation Testing – Check the system against any agreements made. (Done by clients with help from developers, if needed).

More Definitions…

- A Component is the class, the cluster of classes, the package or one or more subsystems that is being tested.
- A Test Case is a set of inputs and expected results that exercises a component with the purpose of causing failures and detecting faults.
- A Test Stub is partial implementation of a component upon which the component being tested depends.
- A Test Driver is a partial component that depends on the component being tested.
  - – Stubs and Drivers allow the component to be *harnessed* – isolated from the rest of the system.

More Definitions…

- A Correction is a change to a component made to repair a fault. *A correction can introduce new faults…*

- A system that fails after a correction has *regressed*.

- Carrying out the previous set of integration tests again is called Regression Testing.

Unit Testing

Unit Testing

- Common types of Unit Testing:
  - Equivalence testing
  - Boundary testing
  - Path testing
  - State-based testing

Equivalence Testing

- A Blackbox technique.– (Deal with the input/output of the component, leave the internal behaviour alone. For now…)
- Minimal number of test cases.

- First, determine a set of equivalence classes, i.e. form groups of test inputs of similar behaviour or nature.
- You assume that the component behaves the same for all members of the equivalence class.

Equivalence Testing, Cont.

- Criteria for equivalence classes:
  - *Coverage* – every possible input would belong to one of the equivalence classes.
  - *Disjointedness* – no input belongs to more than one equivalence class.
  - *Representation* – if one member of an equivalence class causes an erroneous state, then the same state can be caused by any other member of that class.

- For each class, select at least two test inputs – a valid input and an input that should exercise your exception handlers.

Equivalence Testing, Cont.

- Example – suppose you are testing a class that stores dates, as given by a year and month.
- You would need six equivalence classes:
  - 31 day months, non-leap years
  - 31 day months, leap years
  - 30 day months, non-leap years
  - 30 day months, leap years
  - 28 day month, non-leap years
  - 29 day month, leap year
- Minimum of 12 tests!

Aside: Leap Year or Not

How to determine whether a year is a leap year

To determine whether a year is a leap year, follow these steps:

1. If the year is evenly divisible by 4, go to step 2. Otherwise, go to step 5.
2. If the year is evenly divisible by 100, go to step 3. Otherwise, go to step 4.
3. If the year is evenly divisible by 400, go to step 4. Otherwise, go to step 5.
4. The year is a leap year (it has 366 days).
5. The year is not a leap year (it has 365 days).

Boundary Testing

- Look at the "edges" of the equivalence classes.

- For example:
  - Leap year divisible by 400 (year 2000, for example)
  - Non-leap year divisible by 100 (year 1900, for example)
  - Invalid month (at negative edge of legal months): 0
  - Invalid month (at positive edge of legality): 13

Equivalence and Boundary Testing

- The problem with these techniques is that they do not explore many combinations of input data.

- Often a strange combination will cause an error, and this will take many more tests to locate.

Path Testing

- A Whitebox technique.
  - (Focus on the internal structure of the component. Test every state of the component and every possible interaction of objects.)
- Exercise every possible path through the code at least once.
  - Exercise every branch of every if statement.
  - Run each loop – can you cause a loop not to run?
  - Throw each exception.

- A code coverage tool (*like the one in Eclipse*) can help you to confirm that you have tested everything at least once.

Path Testing, Cont.

- The problem with this technique is that it cannot show where code is missing – it only tests existing code.

State-based Testing

- Developed for GUI systems.
- A newer technique that is not fully fleshed out. (*And, the tools can be expensive!*)

- Based on the idea that objects (like a button) can have different states under different conditions (like active, inactive, pressed, mouseover, etc.)

- See Wikipedia entry on "List of GUI Testing Tools".

Polymorphism Testing

- OOP provides additional challenges to testing when a program takes advantage of polymorphism.
- You must make sure that every state of an object is tested.
- This can lead to many more tests especially when polymorphism is used by several objects in a component.

Integration Testing

Integration Testing

- Carried out after unit testing is complete.

- Start with the smallest combinations of components possible, and then let the groups get larger as testing proceeds.

- Still need drivers and stub code. (Note that Extreme Programming, or "XP", requires that all drivers are written before components are developed!).

Integration Testing, Cont.

- Four strategies:
  - Big Bang testing
  - Bottom-Up testing
  - Top-Down testing
  - Sandwich testing

- (*Who thinks up these names…*)

Big Bang Testing

- Once unit testing is complete, test full system.

- You don't need any more drivers, but it is very hard to pinpoint the source of an error.

- (*This is what you are going to do in your projects, isn't it?...*)

Bottom-Up Testing

- Start at the lowest layer (independent classes).
- Combine classes one at a time – double testing, triple testing, quadruple testing, etc.
- No stub code is required, only drivers to simulate the level above the one being tested.

- Sounds nice and safe to me!

Bottom-Up Testing, Cont.

- Interface faults are easily found.

- The problem with Bottom-Up is that UI components are tested last. A correction in this important top layer may require changes all the way back down…

Top-Down Testing

- Opposite to Bottom-Up (*that's a surprise!*)
- Uses stub code for the lower layers, as required. No drivers required.

- Starts with the UI – the most important part of the system.

Sandwich Testing

- Combination of bottom-up and top-down.
- Break the system down into three layers (to form the sandwich). Top, Bottom and Middle.
- Apply both bottom-up and top-down at the same time.
- You will not need drivers and stubs for the outer layers.

- "Modified Sandwich Testing": Test the layers independently and then test all together.

System Testing

Sandwich Testing

- Test the entire system to make sure in complies with the functional and non-functional requirements.

- Carry out:
  - Functional testing
  - Performance testing
  - Pilot testing
  - Acceptance testing
  - Installation testing

System Testing, Cont.

- Functional testing:
  - Also called requirements testing.
  - Compare the system to the functional requirements.
  - Blackbox technique.
  - Tests are extracted from the use case models.
  - The tester should choose the tests that are of the most importance to the user and the most likely to demonstrate a failure.
  - Test cases are created as they were with equivalence and boundary testing.

System Testing, Cont.

- Performance testing:
  - Examine the difference between the system and the design goals.
  - Extract tests from the SDD or the RAD.
  - Examples:
    - Stress testing
    - Volume testing
    - Security testing
    - Timing tests
    - Recovery tests

System Testing, Cont.

- Pilot testing:
  - See "Alpha Testing"
- Acceptance testing:
  - The client prepares benchmark tests.
  - You might test the new system vs. the legacy system.
- Installation testing:
  - The testing of the system after installation.
  - Probably repeat many of the earlier tests.
  - (Can't take too long!)

Post Release Testing

- This brief discussion above was all about pre-release testing.

- Alpha Testing is carried out by a small team of potential customers on site.

- Beta Testing is all the rage these days. Release to the public, but on a somewhat restricted basis. Must have some way of getting feedback from the beta testers in return for getting free software ahead of other people.

Corrections

- What to do when you find a fault?
- How do you reduce the odds of introducing new faults into the repaired component?
- (A big topic!)

- Techniques that can be used:
  - Problem tracking
  - Configuration management
  - Regression testing
  - Rationale maintenance

Some Measures of Software Testing

- Bugs found per tester per unit time (day/week/month).
- Total bugs found in a release.
- Total bugs found in a module or feature.
- Bugs found or fixed per build.
- Number of customer reported bugs.
- *And a lot more …*